

 UNIVERSIDAD DON BOSCO	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN		CICLO 02-2026
	MATERIA	DESARROLLO DE SOFTWARE PARA MOVILES	GUIA DE LABORATORIO #5
	PRÁCTICA	GESTION DE RECURSOS	

I. OBJETIVOS

- Conocer e incorporar diferentes tipos de recursos en Android.
- Desarrollar aplicación Android con acceso a varios recursos.

II. INTRODUCCIÓN TEORICA

Recursos

Los recursos son los archivos adicionales y el contenido estático que usa el código de la App, como mapas de bits, definiciones de diseño, strings de interfaz de usuario, instrucciones de animación, entre otros.

Siempre se debe externalizar los recursos para aplicaciones, como imágenes y strings en el código, para mantenerlos de forma independiente. También se debe proporcionar recursos alternativos para configuraciones de dispositivos específicos, agrupándolos en directorios de recursos con un nombre especial.

En tiempo de ejecución, Android utiliza el recurso adecuado según la configuración actual. Por ejemplo, se puede proporcionar un diseño de interfaz de usuario (IU) diferente según el tamaño de la pantalla o strings diferentes según la configuración de idioma.

Una vez que externalizas los recursos para la aplicación, se puede acceder a ellos mediante los ID de recursos que se generan en la clase R del proyecto.

Tipos de recursos

Android posee 2 grandes tipos de recursos:

- **Recursos del proyecto:** son recursos que son exclusivos del proyecto donde residen (recursos alternativos)
- **Recursos del sistema:** son parte del sistema operativo y mantienen compatibilidad con las diferentes versiones de este.

Recursos del proyecto.

Debes colocar cada tipo de recurso en un subdirectorio específico del directorio res/ de tu proyecto. Por ejemplo, esta es la jerarquía de archivos de un proyecto simple:

```
MyProject/
  src/
    MainActivity.java
  res/
    drawable/
      graphic.png
    layout/
      main.xml
      info.xml
    mipmap/
      icon.png
    values/
      strings.xml
```

El directorio **res/** contiene todos los recursos (en subdirectorios), un recurso de imagen, dos recursos de diseño, directorios **mipmap/** para los iconos del selector y un archivo de recursos de strings.

Los nombres del directorio de recursos son importantes y se describen la siguiente tabla.

Carpeta (identificado r con clase R)	Descripción breve
res/ drawable R.drawable	Archivos de mapas de bits (.png, .png, .jpg, .gif) o archivos XML que se han compilado en los siguientes subtipos de recursos de elemento de diseño: Archivos de mapas de bits, Nine patches (mapas de bits reajustables), Listas de estados, Formas, Elementos de diseño de animaciones, Otros elementos de diseño
res/layout R.layout	Archivos XML que contienen layouts.
res/menu R.menu	Archivos XML con descriptores de menús que podemos usar en nuestra aplicación.
res/anim R.anim	Archivos XML que permite definir una animación con interpolación de movimientos.
res/ animator R.animator	Archivos XML que permiten modificar las propiedades de objetos a lo largo del tiempo.
res/xml R.xml	Otros archivos XML como archivos de preferencias.
res/raw R.raw	Archivos de Audio, video, texto, etc.
res/values	Archivos XML que definen un determinado valor para referencias un color, estilo, cadena.

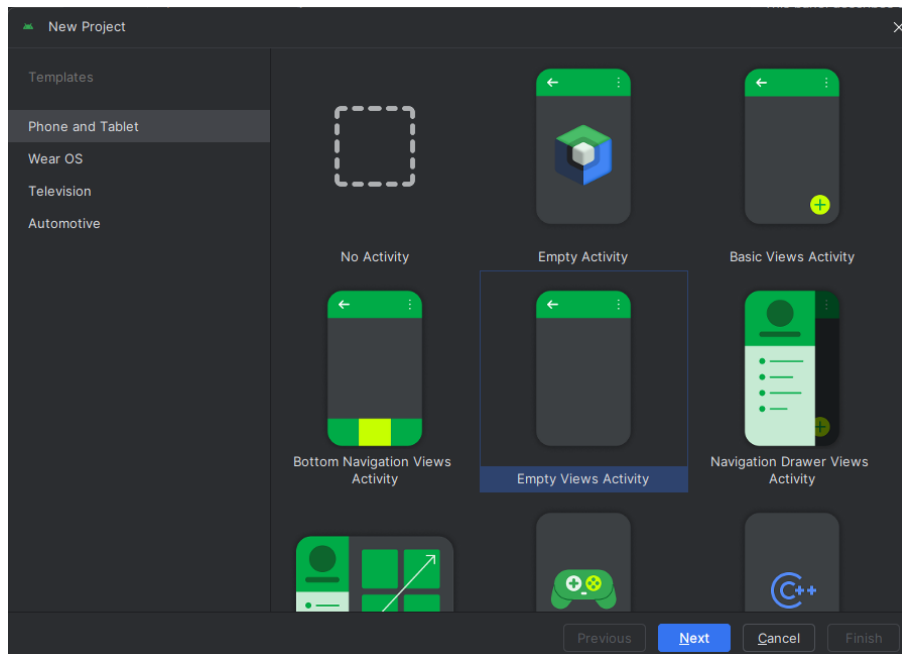
Archivos (identificador)	Descripción breve
strings.xml R.string	Identifica cadenas de caracteres <string name="hola">Hola mundo</string>
colors.xml R.color	Colores definidos en formato ARGV, los valores se indican en hexadecimal en los formatos #RGB, #ARGB, #RRGGBB o #AARRGGBB

	<pre><color name="verde_opaco">#0f0</color> <colo rname="Rojo_translucido">#80ff0000</color></pre>
dimensions.xml R.dimen	Un número seguido de una unidad de medida px - pixeles, mm - milímetros, in - pulgadas, pt - puntos dp - pixeles independientes de la densidad. <pre><dimen name="alto">2.2mm</dimen> <dimen name="tamaño_fuente">16dp</dimen></pre>

Archivos (identificador)	Descripción breve
styles.xml R.styles	Serie de atributos que pueden ser aplicados a una vista o actividad. Si se aplican a una actividad se conocen como temas. <pre><style name="TextoGrande" parent="@style/Text"> <item name="android:textSize">20pt</item> <item name="android:textColor">#000080</item> </style></pre>
R.int	Define un valor entero <pre><integer name= "edad">20</integer></pre>
R.bool	Define un valor booleano <pre><bool name= "continuar">true</bool></pre>
R.id	Define un identificador único de recurso. <pre><item type="id" name="boton_ok" /> <item type="id" name="txtNombre" /></pre>
R.array	Una serie ordenada de elementos. Pueden ser strings, enteros o de recursos (TypedArray) <pre><string-array name= "dias_semana"> <item>lunes</item> <item>martes</item> </string-array> <integer-array name= "meses"> <item>1</item> <item>2</item><item>3</item> </integer-array> <array name= "fotos"> <item>@drawable/foto1</item> <item>@drawable/foto2</item> </array></pre>

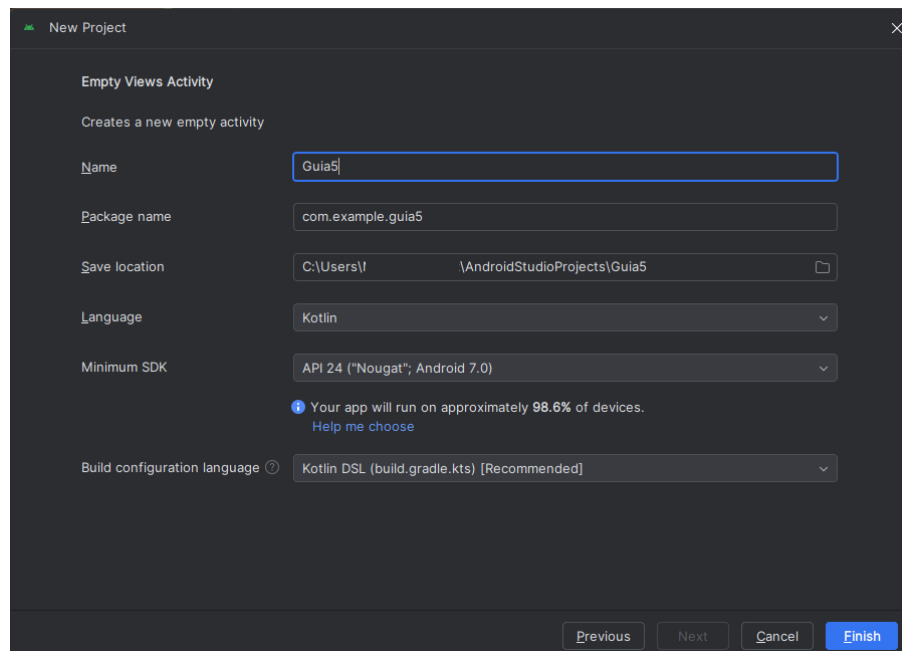
III. DESARROLLO DE LA PRÁCTICA

1. Cree un nuevo proyecto en Android Studio, asegúrese de seleccionar la plantilla **Empty Views Activity**.

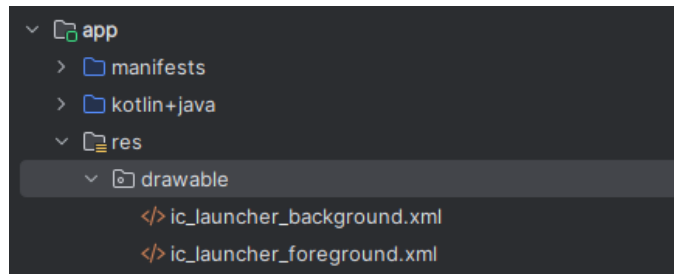


2. Configure el proyecto con las siguientes características:

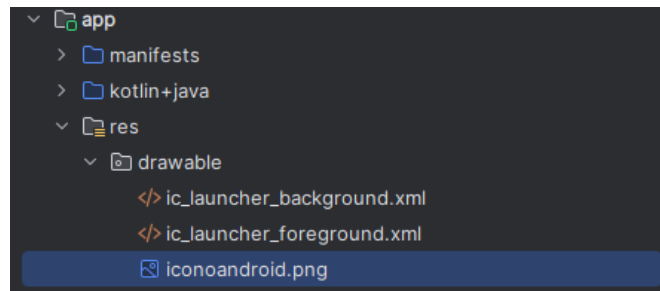
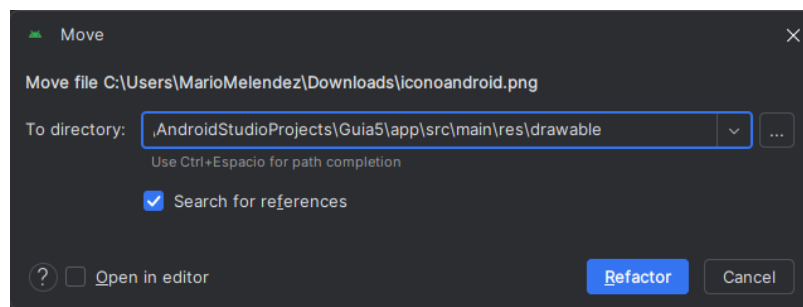
- **Nombre:** Guia5.
- **Ubicación:** por defecto (o use una ubicación de su preferencia).
- **Lenguaje:** Kotlin.
- **Minimun SDK:** API 24 Nougat.



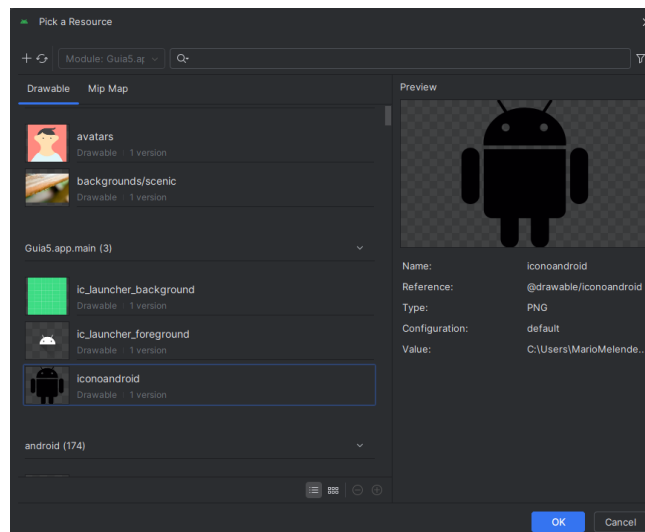
3. En la carpeta **res/drawable** arrastre una imagen cualquiera en formato png con identificador **iconoandroid.png**



4. Aparecerá la siguiente ventana para confirmar el copiado de la imagen en la carpeta **drawable**.



5. Agregue un **ImageView** al **activity_main.xml**, en la vista Design de esta activity. Luego, asigne la imagen recién copiada de los recursos al atributo **srcCompat**.



6. Haga clic en **Ok** para finalizar.

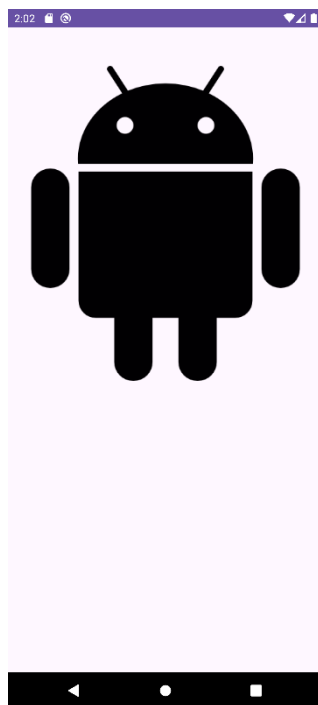
7. Modifique el código del **activity_main.xml** como se muestra a continuación, asegurarse que la vista de tipo **ImageView** tenga el id **imageView** de la siguiente manera **android:id="@+id/imageView"**, posteriormente este recurso "imageView" será buscado en el archivo **R.txt**.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

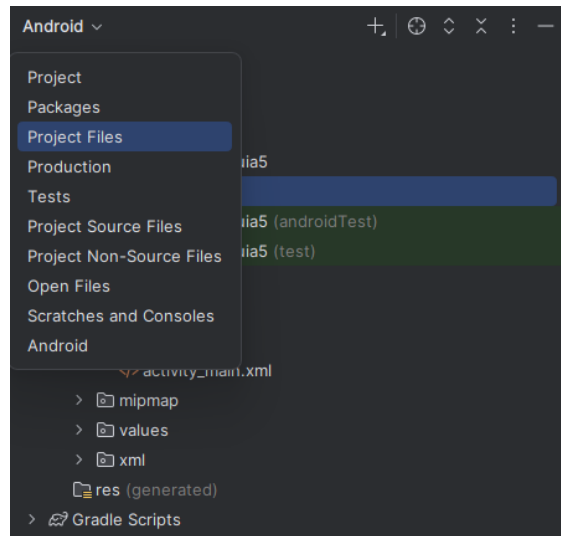
    <ImageView
        android:id="@+id/imageView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:srcCompat="@drawable/iconoandroid" />

</LinearLayout>
```

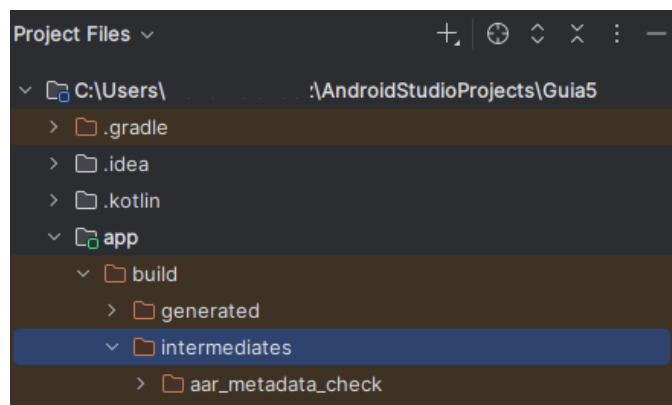
8. Ejecute la aplicación y visualice el resultado.



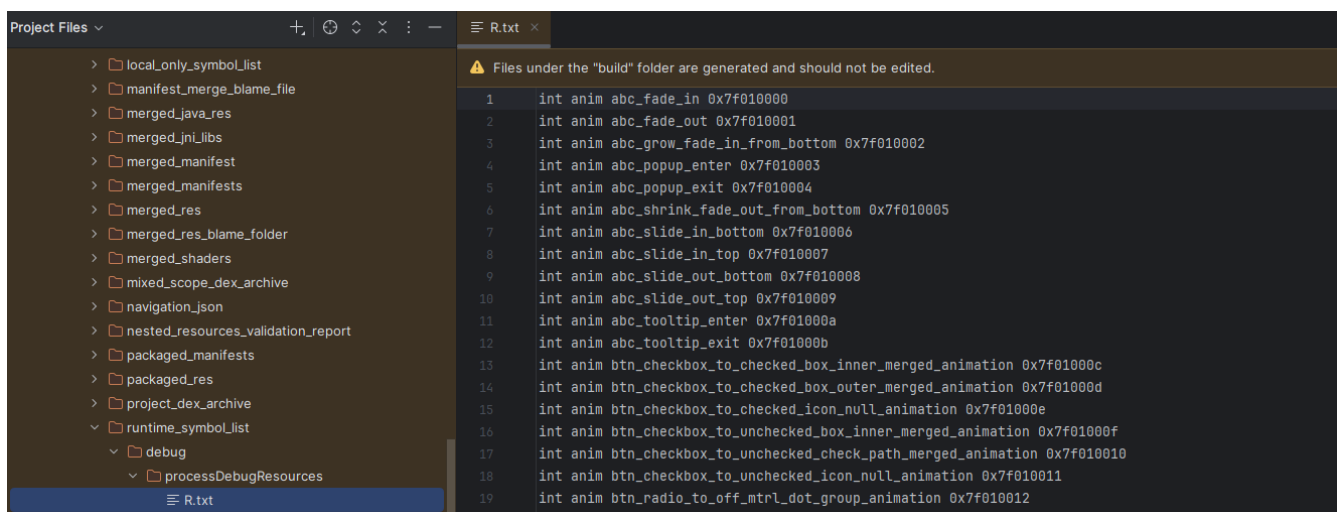
9. Desde el panel izquierdo **Project** de **Android Studio**, cambie la vista de la estructura del proyecto de **Android** a **Project Files** como muestra la siguiente figura.



10. Localice el directorio **app/build/intermediates**.



11. Dentro del directorio **intermediates**, localice archivo **R.txt** el directorio **runtime_symbolist/debug/processDebugResources**.



12. Dentro del archivo **R.txt**, cuando el archivo **R.txt** este abierto, busque el recurso con identificador **imageView**. El objetivo es verificar que efectivamente cualquier recurso creado y gestionado en la aplicación Android, debe aparecer en el archivo **R.txt** con un identificador numérico en formato **hexadecimal**, para poder hacer referencia a este o cualquier otro recurso posteriormente en el código Kotlin del mismo proyecto Android.

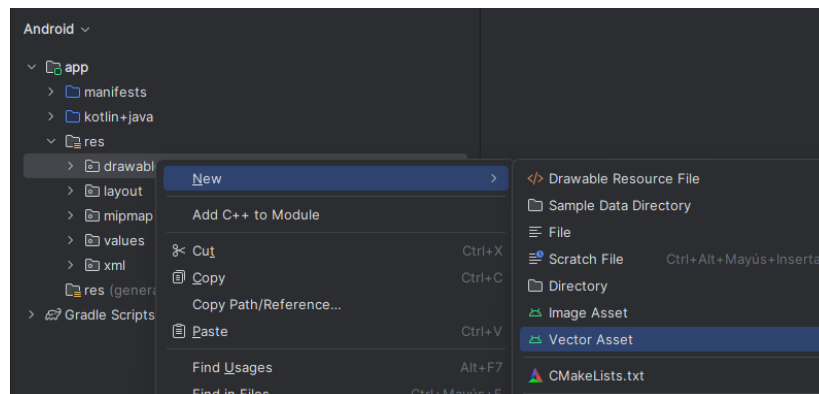
```
int id image 0x7f0800e0
int id imageView 0x7f0800e1
int id immediateStop 0x7f0800e2
int id included 0x7f0800e3
```

13. Busque el recurso **activity_main** y la imagen agregada al ImageView con identificador **iconoandroid**, compruebe el identificador numérico.

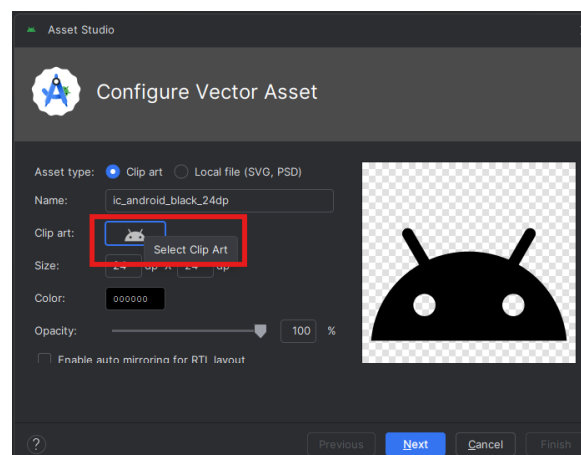
```
int layout abc_tooltip 0x7f0b001b
int layout activity_main 0x7f0b001c
int layout custom_dialog 0x7f0b001d

int drawable ic_search_black_24 0x7f07009a
int drawable iconoandroid 0x7f07009b
int drawable indeterminate_static 0x7f07009c
int drawable m3_avd_hide_password 0x7f07009d
```

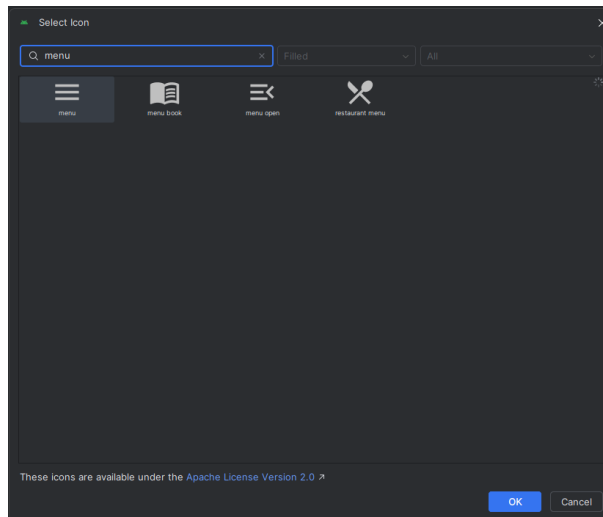
14. Regrese a la vista de proyecto **Android** y luego en la carpeta **res/drawable** haga clic derecho y cree un nuevo recurso seleccionando **New -> Vector Asset**.



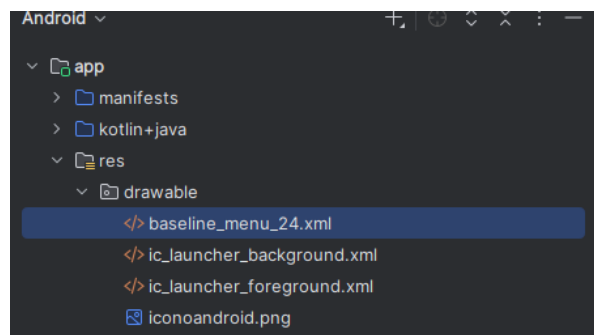
15. Haga clic en el botón **Select Clip Art**



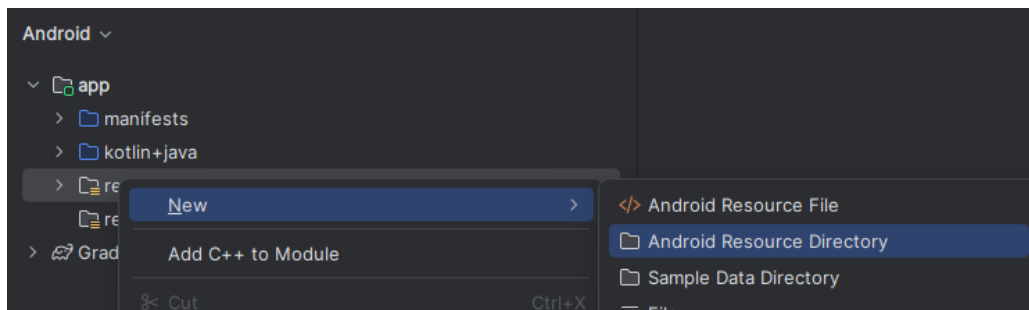
16. En la ventana **Select Icon**, busque el icono menú, como muestra la siguiente figura. Una vez encontrado, haga clic en **Ok**.



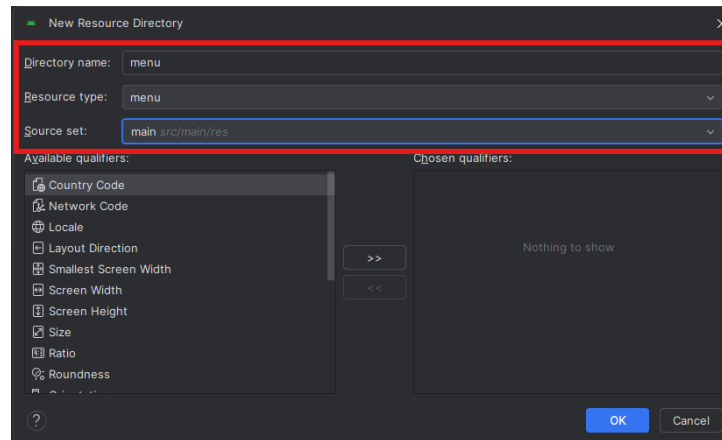
17. En la ventana **Asset Studio** haga clic en **Next** y posteriormente **Finish** para guardar el recurso en la carpeta **drawable**.



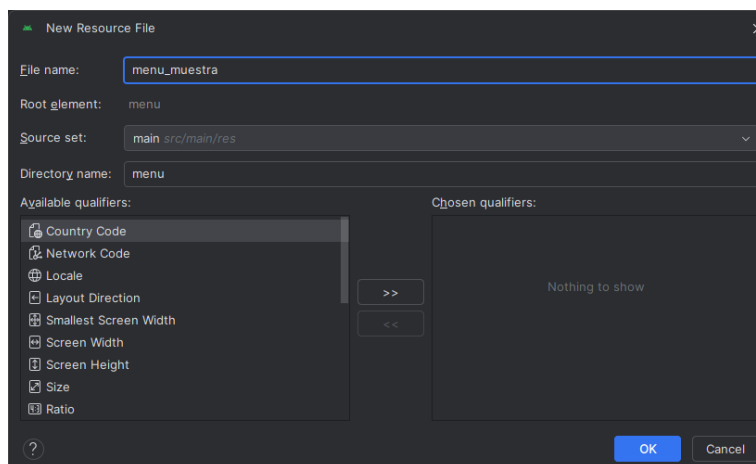
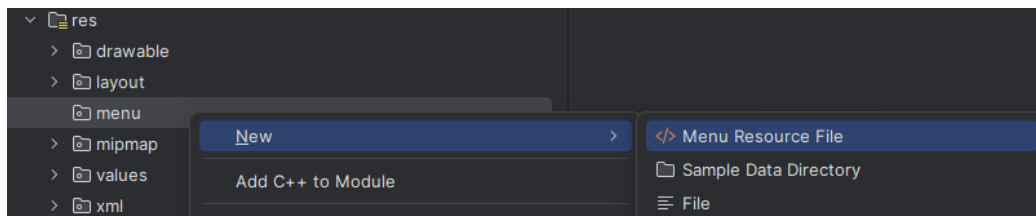
18. En la carpeta **res** haga clic derecho y cree un nuevo directorio de recursos seleccionando **New -> Android Resource Directory**.



19. Configure el nuevo directorio con los valores que se muestran a continuación y haga clic en **OK**.



20. Sobre el directorio **res/menu** recién creado, haga clic derecho y cree un nuevo recurso de tipo **Menu Resource File** con el nombre **menu_muestra**, luego, haga clic en **OK**.



21. Modifique el código del archivo **menu_muestra.xml** de la siguiente manera.

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item android:id="@+id/menu1"
        android:icon="@drawable/baseline_menu_24"
        android:title="Menu"
        app:showAsAction="ifRoom" />
    <item android:id="@+id/opcion1"
        android:title="Opcion 1"
        app:showAsAction="never" />
</menu>
```

```

<item android:id="@+id/opcion2"
    android:title="Opcion 2"
    app:showAsAction="never" />
<item android:id="@+id/opcion3"
    android:title="Opcion 3"
    app:showAsAction="never" />
<item android:id="@+id/opcion4"
    android:title="Opcion 4"
    app:showAsAction="never" />

</menu>

```

- El atributo **showAsAction** de cada item del menú, indica como mostrara cada opción del menú dependiendo el valor que se configure:
 - **never**: nunca mostrara la opción directamente en la barra **ToolBar** o **ActionBar**.
 - **always**: siempre mostrara la opción directamente en la barra **ToolBar** o **ActionBar**.
 - **ifRoom**: muestra el item directamente en la barra si hay espacio; si no, se coloca dentro del menú.

22. Modifique el condigo del archivo **MainActivity.kt** de la siguiente manera.

```

package com.example.guia5

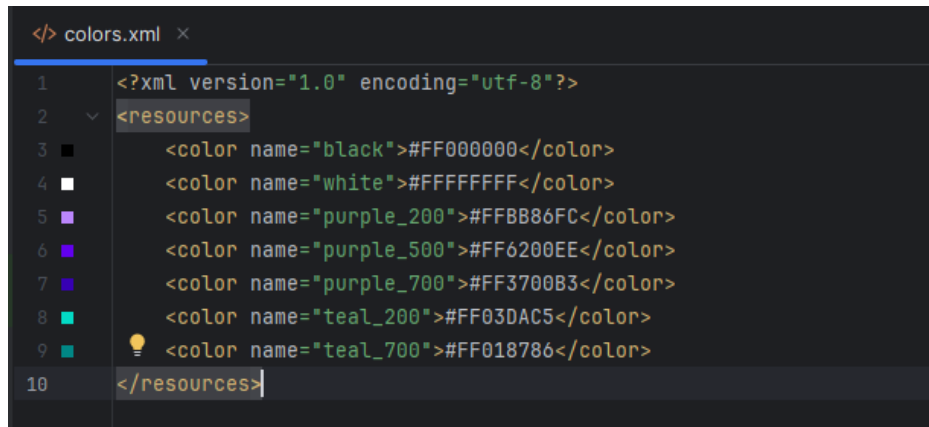
import android.os.Bundle
import android.view.Menu
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

    override fun onCreateOptionsMenu(menu: Menu?): Boolean {
        val inflater = menuInflater
        inflater.inflate(R.menu.menu_muestra, menu)
        return true
    }
}

```

23. Modifique el archive **colors.xml** ubicado en el directorio **res/values**.



24. Modifique el archivo **themes.xml** de la siguiente forma:

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Guia5" parent="Theme.MaterialComponents.DayNight.DarkActionBar">

        <!-- Primary brand color -->

        <item name="colorPrimary">@color/purple_500</item>
        <item name="colorPrimaryVariant">@color/purple_700</item>
        <item name="colorOnPrimary">@color/white</item>

        <!-- Secondary brand color -->

        <item name="colorSecondary">@color/teal_200</item>
        <item name="colorSecondaryVariant">@color/teal_700</item>
        <item name="colorOnSecondary">@color/black</item>

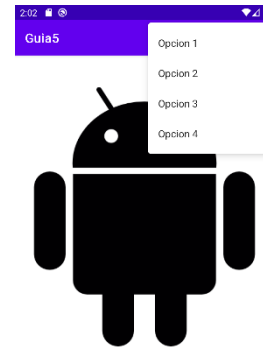
        <!-- Status bar color -->
        <item name="android:statusBarColor">?attr/colorPrimaryVariant</item>

    </style>

    <style name="Theme.Guia5" parent="Base.Theme.Guia5" />
</resources>
```

- Solo modifique la propiedad **parent** superior con el valor **Theme.MaterialComponents.DayNight.DarkActionBar**
- **NO** modifique la propiedad **name** de **style** superior.
- Agregue las etiquetas xml de tipo **item** dentro de **style** superior.
- **NO** modifique las etiquetas **style** inferiores.

25. Ejecute la aplicación y compruebe los resultados.



26. Para agregar comportamiento a la opción de menú, modifique el código de la clase **MainActivity** de la siguiente forma.

```
package com.example.guia5

import android.os.Bundle
import android.view.Menu
import android.view.MenuItem
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import kotlin.time.Duration

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

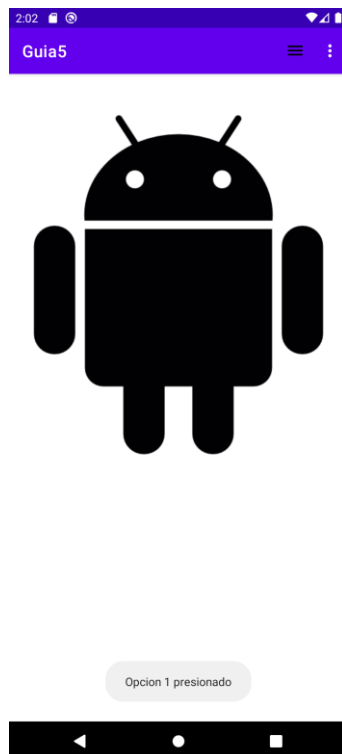
    override fun onCreateOptionsMenu(menu: Menu?): Boolean {
        val inflater = menuInflater
        inflater.inflate(R.menu.menu_muestra, menu)
        return true
    }
}
```

```

override fun onOptionsItemSelected(item: MenuItem): Boolean {
    return when (item.itemId) {
        R.id.opcion1 -> {
            Toast
                .makeText(
                    applicationContext,
                    "Opcion 1 presionado", Toast.LENGTH_LONG)
                .show()
            true
        }
        else -> super.onOptionsItemSelected(item)
    }
}

```

Se sobre escribe el método **onOptionsItemSelected** para agregar la lógica según sea la opción seleccionada, para este ejemplo únicamente está configurada la opción 1 del menu.



IV. DISCUSIÓN DE RESULTADOS

Utilizando el mismo proyecto cree 4 nuevas actividades y configure las opciones de menú para que cuando el usuario seleccione una opción se abra la actividad correspondiente, ejemplo: Opción 1 -> Activity 1, Opción 2 -> Activity 2, etc. Además, agregue una opción de regresar a la actividad principal.

V. BIBLOGRAFÍA

- Documentación Oficial Android - Descripción general sobre los recursos de las apps.
<https://developer.android.com/guide/topics/resources/providing-resources?hl=es-419>